

シミュレーション レポート 4

神野 友哉

2022311042

愛知県立大学 情報科学部 情報科学科

2024 年 5 月 15 日

目 次

1 実験項目 1	2
1.1 概要	2
1.2 目的	2
1.3 方法	2
1.4 結果	4
1.5 考察	7
1.6 感想	7
2 実験項目 2	7
2.1 概要	8
2.2 目的	8
2.3 方法	8
2.4 結果	9
2.5 考察	11
2.6 感想	11
3 実験項目 3	11
3.1 概要	11
3.2 目的	11
3.3 方法	11
3.4 結果	12
3.5 考察	14
3.6 感想	14
4 実験項目 4	14
4.1 概要	15
4.2 目的	15
4.3 方法	15
4.4 結果	15
4.5 考察	17

1 実験項目 1

各変数 (m , h , n) の初期値は、 $V = -65[mV]$ とした際の定常状態の値を設定する。時刻 $100[ms]$ までシミュレーションして初期値を求めよ、得られた初期値をそれぞれの Integrator の初期条件に入力する。

1.1 概要

目的通りに、シミュレーションによって定常状態の出力値を求められたため、それを Integrator に初期状態として与えることが可能となった。また、常微分方程式の解法より、シミュレーションを用いない方法で、定常状態の値を桁切り捨て込みで、ある程度の精度 (誤差 $O(e^{-4})$) で求める方法を確認できた。余談だが、ブロック線図の構築や式の理解を深めることが十分できたと考えられ、この実験のペースは堅調であった。

1.2 目的

定常状態の値を初期状態に与えれば、経過の余分な推移を避けられると考えられるため。

1.3 方法

- 手順 1 step を置き、 -65 を入れる。縦並びに 40、65、65、35、55、65 と入力した constant を配置し、その出力を入力としてとるように各々 add に繋ぐ。それらの add もう一方の入力は、先ほどの step の出力を分岐させて接続する。
- 手順 2 一番上、40 の add の出力に、 -0.1 の gain を繋ぎ、その出力を分岐させる。一方を exp へ繋ぎ、sum と constant を用いて 1 を引く。もう一方を divide の $\times$ に繋ぎ、exp 側に分岐した線の最後尾を $\div$ に繋ぐ。
- 手順 3 手順 2 の divide の先に図 1 を作成する。この時、手順 2 の出力は、図 1 の上側の入力とする。また、Integrator には 0 と入れる。
- 手順 4 縦並び constant の上から 2 番目 65 の add の先に divide を置き、divide は $\times$ に繋ぎ、 -18 を入れた constant の出力は $\div$ に繋ぐ。その出力を exp に繋ぎ、さらに 4 の gain に繋ぎ、手順 3 の図 1 の下側の入力とする。
- 手順 5 手順 4 と同様に、上から 3 番目 65 の add の先を作成する。異なるのは、途中の constant が -20 であることと、gain が 0.07 であることである。また、手順 3 と同様に、その出力の先に図 1 を作成し、それを上側の入力に接続する。
- 手順 6 上から 4 番目 35 の add の先に -0.1 の gain を置き、接続した上で、exp の入力に繋ぐ。その出力を sum と constant を用いて 1 を足し、 $\times$ に 1 が入る divide の $\div$ に繋ぎ逆数にする。その出力を、手順 5 の図 1 の下側の入力とする。

- 手順7 上から5番目55の add の先を作成する。手順2と同様であるが、もう一方への divide の $\times$ 入力に接続する線の間に、0.1 の gain を置く点で異なる。その divide の出力は手順3と同様に図1を作成する。そして、上側の入力に接続する。
- 手順8 一番下、上から6番目65の add の先を作成する。手順4と同様であるが、constant に -80、gain に 0.125 を入れ、gain の出力を手順7の図1の下側の入力とする。
- 手順9 display を3つ置き、Integrator の出力にそれぞれ対応させる。図の scope、display が上から、m、h、n に対応している。また、scope も3つ、それぞれの Integrator の線を分岐させて接続する。そして、終了時間を100に設定し、実行する。

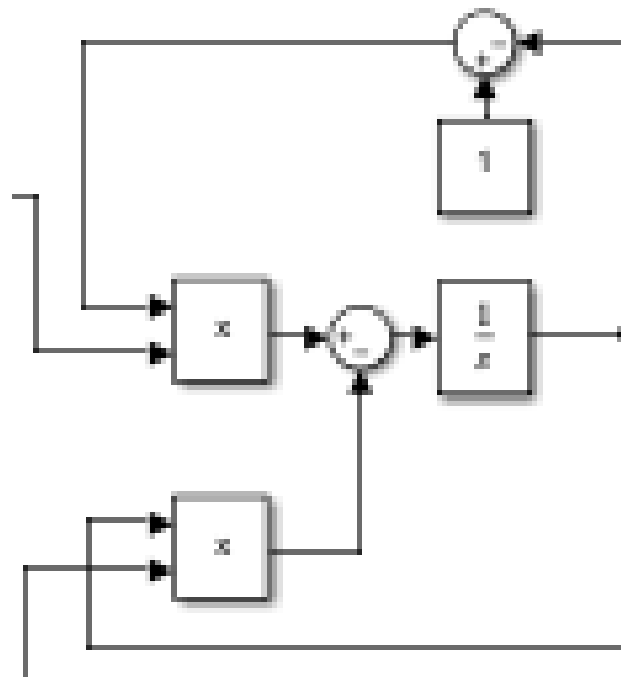


図 1: 積分器部分

以下の図2のような、ブロック線図が作成されるはずである。

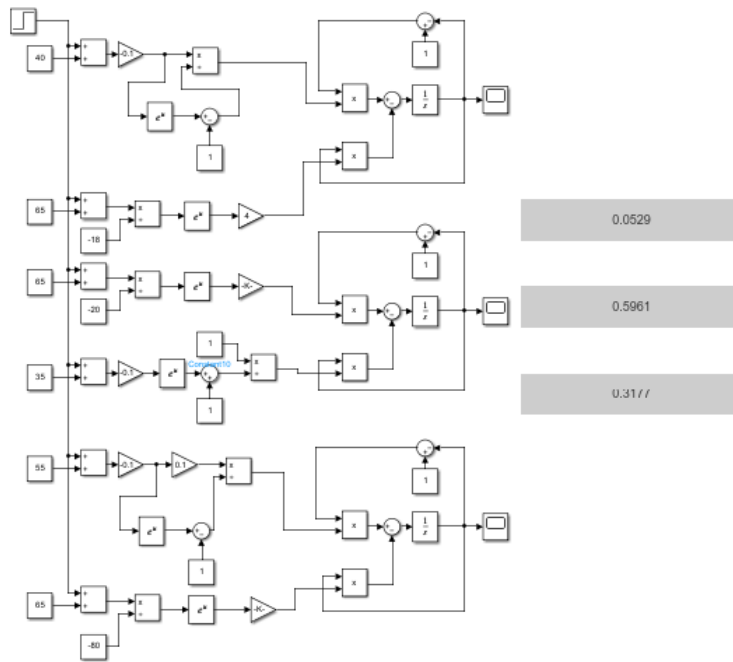


図 2: 作成したブロック線図

さらに、下図3はHodgkin-Huxley モデル全体のブロック線図である。求めた初期値をこれらの Integrator に入れる。この図3の通りに改造する。詳しくは実験項目2の方法部分で作成過程を書いている。

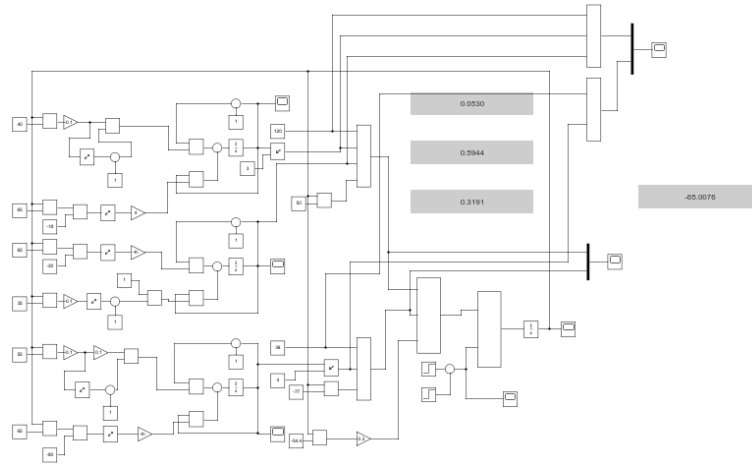


図 3: モデル全体ブロック線図

1.4 結果

図2を実際に実行した結果、display から、 m 、 h 、 n がそれぞれ、0.0529、0.5961、0.3177 となったと判った。また、図4、図5、図6は、 m 、 h 、 n が上記値に収束していることを示している。

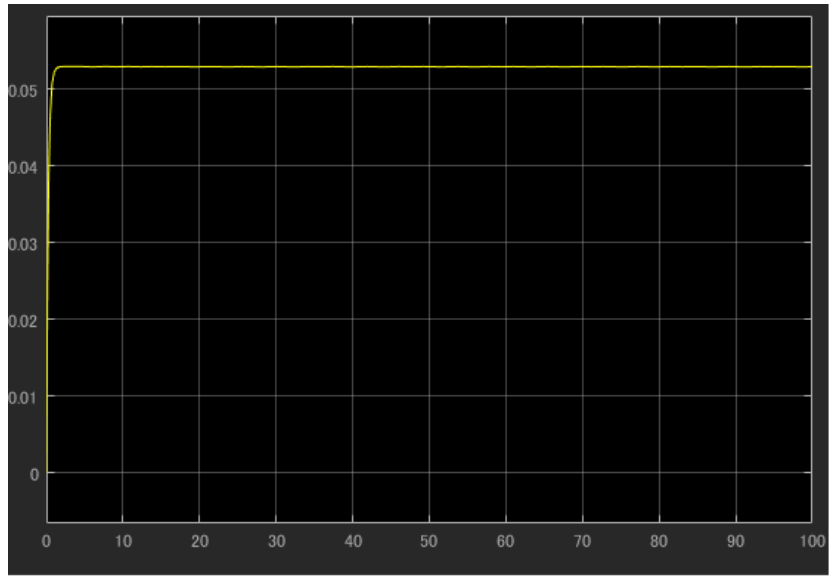


図 4: m の収束過程

また、図 3 の Integrator に m 、 h 、 n の各値を入れた。

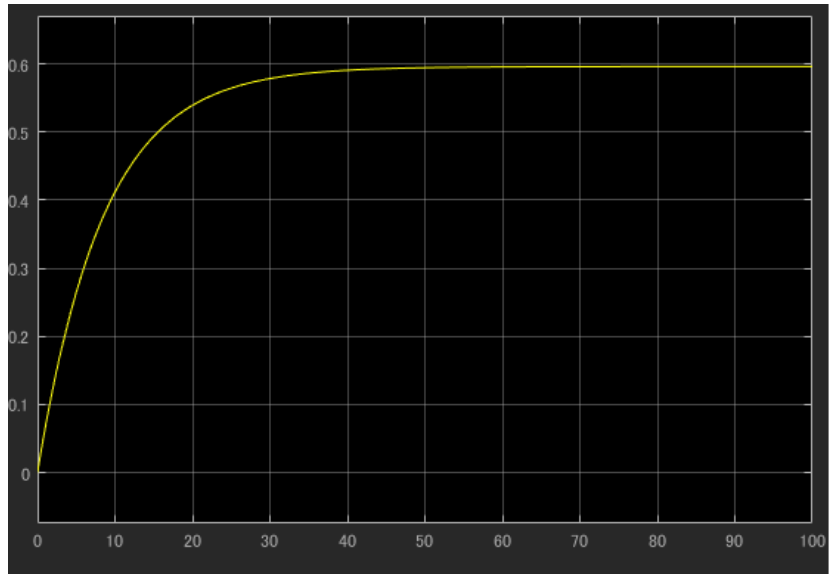


図 5: h の収束過程

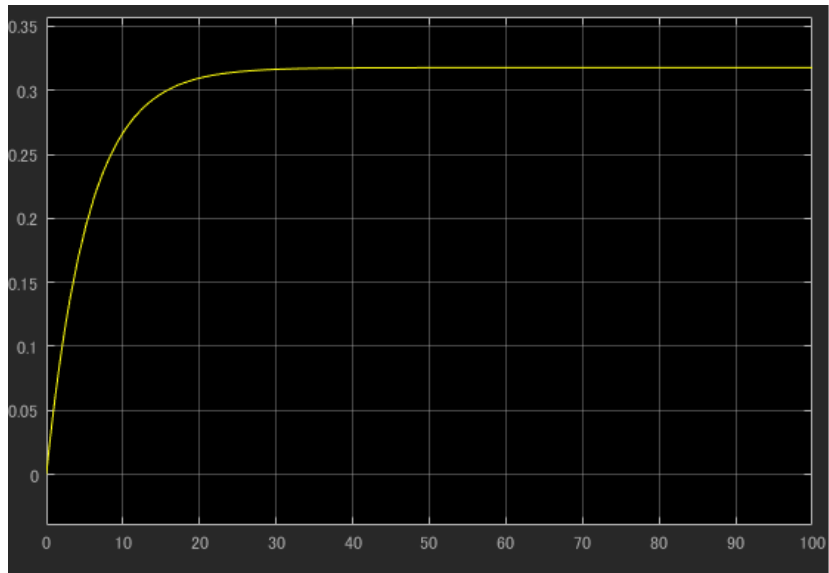


図 6: n の収束過程

1.5 考察

図 4、図 5、図 6 から、このブロック線図によるシミュレーションが特定の値に収束させていることは自明であり、ブロック線図自体が数学的な式の組み合わせによってのみ成り立っている。そのため、図 1 の積分器 Integrator ループ部分の値が常微分方程式の解を求めるには、 t について積分し、 m 等について解くと求まると考えられる。具体的には、

$$\frac{dm}{dt} = \alpha_m(1 - m) - \beta_m m \quad (1)$$

$$\frac{dm}{dt} = \alpha_m - (\alpha_m + \beta_m)m \quad (2)$$

$$m = \{\alpha_m - (\alpha_m + \beta_m)m\}t \quad (3)$$

$$(1 + (\alpha_m + \beta_m)t)m = \alpha_m t \quad (4)$$

$$m = \frac{\alpha_m t}{1 + (\alpha_m + \beta_m)t} \quad (5)$$

$$\text{ここで、} V = -65 \text{ を } \alpha_m, \beta_m \text{ の式に代入すると、} \quad (6)$$

$$\alpha_m = \frac{-0.1(-65 + 40)}{e^{\frac{-(-65+40)}{10}} - 1} \quad (7)$$

$$= \frac{2.5}{e^{2.5} - 1} \simeq 0.223563 \quad (8)$$

$$\beta_m = 4e^{-\frac{-65+65}{20}} \simeq 4 \quad (9)$$

$$\text{よって、} m = \frac{0.223563t}{1 + 4.223563t} \quad (10)$$

$$t = 100(\text{終了時間}) \text{ とすると、} m = \frac{22.3563}{1 + 422.3563} \simeq 0.052807 \quad (11)$$

$$t = 1000(\text{終了時間}) \text{ とすると、} m = \frac{223.563}{1 + 4223.563} \simeq 0.052919 \quad (12)$$

$t = 100$ の際の m の誤差が e^{-4} の桁で存在していることから、計算上の切り下げや機械的な丸め誤差で数値解析的な解とシミュレーションに違いが生じていると考えられる。しかし、十分な精度ではあるとも考えられるのではないか。また、この方法で同様に h 、 n も求められる。

- 初期値を理論的に求める方法 求めたいパラメータ (便宜上 λ とする) の常微分方程式を t について積分し、 λ について解く。また、 V を $\alpha_\lambda, \beta_\lambda$ に代入した値を可能な限り正確に出す。それを先ほどの式に代入し、終了時間として与える t も代入した値を極力正確に出す。そうして、 λ の正確な値が求まる。

1.6 感想

ブロック線図を作成するために掛けた時間は、確認込みで 40 分程度で、資料のブロック線図を step の部分を除いて見ずに作成したため、今までの実験を通して matlab の simulink の理解度が高まっていると感じた。

2 実験項目 2

入力刺激電流 I_{ext} が時刻 $2[ms]$ において、 $0[\mu A/cm^2]$ から $5[\mu A/cm^2]$ に $2[ms]$ 間、パルス状に変化した場合の膜電位変化、各イオン電流、各コンダクタンス変化を観測、プロットせよ。なお、入力刺激用のパルスはステップ関数を 2 つ組み合わせて生成し、意図した形状になっていることを確認せよ。

2.1 概要

実験項目 1 の図 3 を用いて実験した。資料に載っていたグラフと概ね同様な結果となったため、シミュレーションとして正当性のあるブロック線図が構築できたと結論付けた。また、入力刺激用のパルス信号は意図したものとなっていたことも確認できた。

2.2 目的

モデル全体で各イオン電流や各コンダクタンスが全体の膜電位変化に対し、どのように変化するか観察し考察する。

2.3 方法

手順 1 図 2 の step を消す。

手順 2 m に対応する Integrator の線を分岐し、pow の u に接続、一方 v には 3 の constant の出力を入力する。入力 4 の product を配置し、その入力に m の pow の出力、h、120 の constant の出力、50 の constant を入力に取った add の出力を繋ぐ。手順 1 と手順 2 の add の片方入力はいずれ V を繋ぐために保留している。この V に繋ぐ側にはマイナスを設定する。

手順 3 手順 2 と同様に n の先を作る。この際に、入力 3 の product で pow の v には 4 の constant、36 の constant、add の入力には -77 の constant である点で異なっている。

手順 4 -54.4 の constant を入力に取った add を置き、手順 2、手順 3 同様、もう一方の入力にマイナスを設定し V を繋げるよう保留する。その add の出力に 0.3 の gain を繋ぎ、手順 2、手順 3、手順 4 の出力を入力 3 の add に繋ぐ。

手順 5 step を手順 4 の近くに 2 つ置き、プラスマイナス入力の sum も置く。両方の step で初期値 0、最終値 5 とし、sum プラス側のステップ時間を 2 マイナス側を 4 とする。sum の線を分岐し scope に繋ぐ。

手順 6 手順 5 の sum の出力の先にプラスマイナスの add を置き、プラスに繋ぐ。手順 4 の出力はマイナスに繋ぐ。その add の出力を Integrator に繋ぎ、初期状態に -65 を入れる。この出力の線を全ての保留した入力に分岐させて繋ぐ。

手順 7 手順 2、手順 3 の product の出力を分岐させて入力 2 の Mux に繋ぎ、その出力を scope に繋ぐ。

手順 8 手順 2、手順 3 の保留させた入力に繋いだ線以外を分岐させそれぞれ入力 3、2 の product の入力として繋ぎ、それらの product の出力を入力 2 の Mux に繋ぎ、その出力を scope に繋ぐ。

手順 9 V の Integrator の線を分岐し scope に繋ぐ。そして終了時間を 30 として実行する。

完成したブロック線図が 3 である。

2.4 結果

実行した結果、step による時刻 t に対する入力刺激用のパルス信号の図 7、時膜電位変化が図 8、各イオン電流変化が図 9、各コンダクタンス変化が図 10 となった。黄色がナトリウム、青がカリウムのそれである。

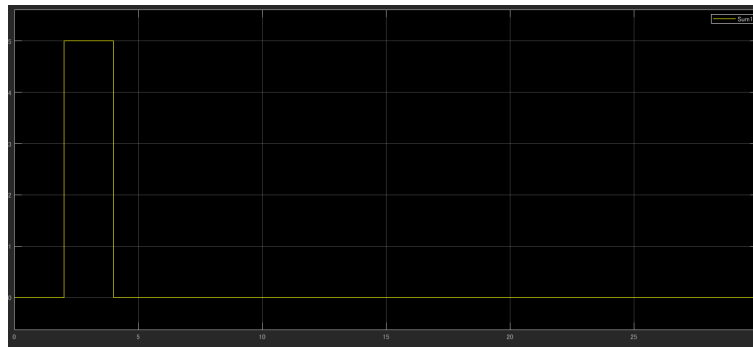


図 7: 入力刺激用のパルス

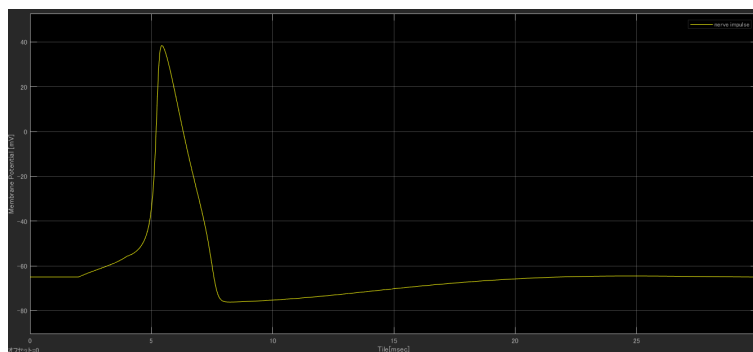


図 8: 膜電位変化

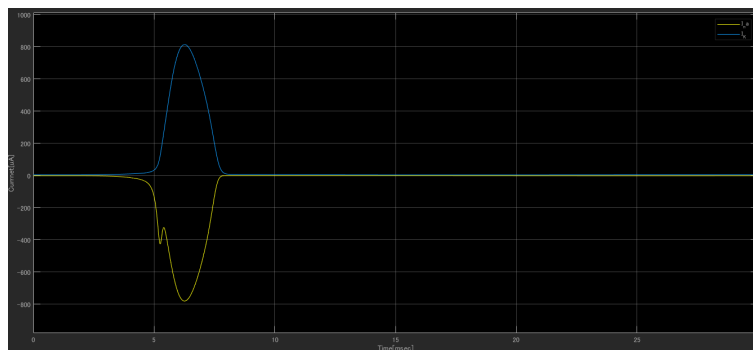


図 9: イオン電流変化

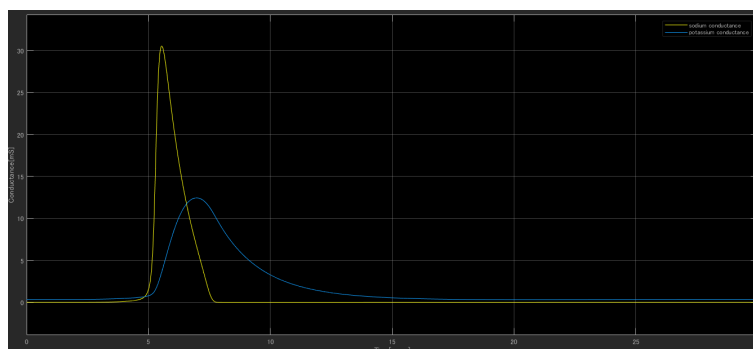


図 10: コンダクタンス変化

2.5 考察

図7のステップに対し、少し遅れて図8のように膜電位変化が起きていることが判る。それは、Integratorの性質から、 t に対する I の曲線と微小な積分区間、 t 軸で囲まれた領域の面積が積分結果として出て微小区間が動くことで領域が推移すること。これが直ちに反応するわけではない理由と考えられるのではないかな。余談だが、その面積の求める方法も数値解析的な近似であり、オイラー法に比べて精度が高いルンゲクッタ法の一つである。

2.6 感想

神経細胞が電荷をもつイオンの領域当たりの数によって信号を伝えていること自体は、トリカブト毒やふぐ毒の薬理作用からある程度理解していたつもりではあるが、やはり猛毒なんだなと感じた。また、サリン等の神経毒は、この神経細胞間の化学的な信号伝達部分に作用することで、神経を麻痺させるという。神経毒を受けた人間の神経系のパルス信号がどのようなのか気になった。

3 実験項目 3

最大コンダクタンスの値 $(\overline{g_{Na}}, \overline{g_K})$ を $\pm 20\%$ の範囲内で種々変化させた場合、パルス状の刺激に対する応答にどのような変化が現れるかを観測せよ。応答の違いがわかるように、シミュレーション結果を重ね描きすること。

3.1 概要

実験項目2で作成したブロック線図を用いて、様々に $(\overline{g_{Na}}, \overline{g_K})$ を変化させたシミュレーションを行い、重ね描きした図から、 $(\overline{g_{Na}}, \overline{g_K})$ の関係性及び、それに伴うシミュレーション結果への影響を考察することが出来た。

3.2 目的

$(\overline{g_{Na}}, \overline{g_K})$ とシミュレーションの結果の関係性を考察するための材料を得る。

3.3 方法

図3の120、36と入れたconstantを前者96、120、144、後者28.8、36、43.2のいずれかを取り、計9通りのシミュレーションを行う。この際に、 V の線を分岐し、simoutでデータを出力し、matlabプログラムで重ね描きplotする。各々実行後に、matlabプログラムのコード1を実行し、datファイルを作成する。全て終わり次第、コード2を実行し、重ね描きplotされたpdfを作成する。

source 1: dat ファイル化コード

```
output = [out.simout.Time out.simout.Data];
na = '6';
si = '0-36';
filename1 = 'kkadai24-gcp-mini-n';
filename2 = 'kkadai24-p-mini-n';

datfilename = [filename1 na 's' si '.dat'];
pngfilename = [filename2 na 's' si '.png'];
pdffilename = [filename2 na 's' si '.pdf'];

save(datfilename, 'output', '-ascii');
SimData = load(datfilename, '-ascii');
plot(SimData(:,1), SimData(:,9), 'Linewidth', 1.5)
xlabel('time [msec]')
ylim([-70 20])
ylabel('cardiac action potential [mV]')
%set(gca, 'YScale', 'log')
legend('output', ...
        'Location', 'southeast', ...
        'Orientation', 'horizontal', 'NumColumns', 1)
grid on 確認用
%png pdf
print('-dpng', pngfilename)
print('-dpdf', pdffilename)
```

source 2: 重ね描き pdf 作成コード

```
SimDataA = load('kkadai13-n96k28-8.dat', '-ascii');
SimDataB = load('kkadai13-n120k28-8.dat', '-ascii');
SimDataC = load('kkadai13-n144k28-8.dat', '-ascii');
SimDataD = load('kkadai13-n96k36.dat', '-ascii');
SimDataE = load('kkadai13-n120k36.dat', '-ascii');
SimDataF = load('kkadai13-n144k36.dat', '-ascii');
SimDataG = load('kkadai13-n96k43-2.dat', '-ascii');
SimDataH = load('kkadai13-n120k43-2.dat', '-ascii');
SimDataI = load('kkadai13-n144k43-2.dat', '-ascii');

figure
plot(SimDataA(:,1), SimDataA(:,2), 'Linewidth', 1.0), hold on
plot(SimDataB(:,1), SimDataB(:,2), 'Linewidth', 1.0)
plot(SimDataC(:,1), SimDataC(:,2), 'Linewidth', 1.0)
plot(SimDataD(:,1), SimDataD(:,2), 'Linewidth', 1.0)
plot(SimDataE(:,1), SimDataE(:,2), 'Linewidth', 1.0)
plot(SimDataF(:,1), SimDataF(:,2), 'Linewidth', 1.0)
plot(SimDataG(:,1), SimDataG(:,2), 'Linewidth', 1.0)
plot(SimDataH(:,1), SimDataH(:,2), 'Linewidth', 1.0)
plot(SimDataI(:,1), SimDataI(:,2), 'Linewidth', 1.0)
xlabel('time [msec]')
ylim([-120 80])
ylabel('Membrane Potential [mV]')
legend('output (gNa = 96, gK = 28.8)', ...
        'output (gNa = 120, gK = 28.8)', ...
        'output (gNa = 144, gK = 28.8)', ...
        'output (gNa = 96, gK = 36)', ...
        'output (gNa = 120, gK = 36)', ...
        'output (gNa = 144, gK = 36)', ...
        'output (gNa = 96, gK = 43.2)', ...
        'output (gNa = 120, gK = 43.2)', ...
        'output (gNa = 144, gK = 43.2)', ...
        'Location', 'northeast', ...
        'Orientation', 'horizontal', 'NumColumns', 3)
set(findobj('type', 'axes'), 'fontsize', 8)
grid on
print -dpdf 'kkadai13(9).pdf'
```

3.4 結果

結果は図 11 となった。判りづらいので、 $(\overline{g_{Na}}, \overline{g_K})$ を固定し 3 通りのシミュレーション結果を重ね書きした図を作成した。図 15 は $\overline{g_{Na}}$ 、図 19 は $\overline{g_K}$ について、それぞれ先述の値に固定した図である。

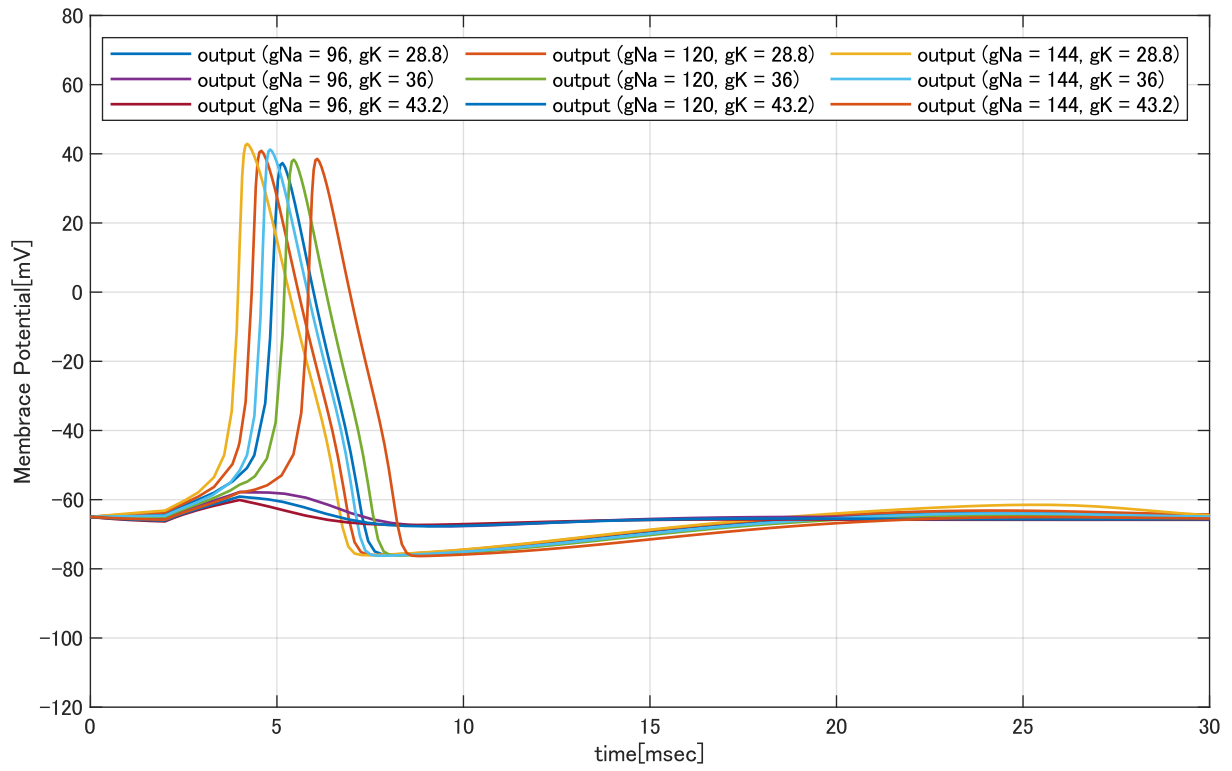


図 11: 9 通りのシミュレーション結果の重ね描き

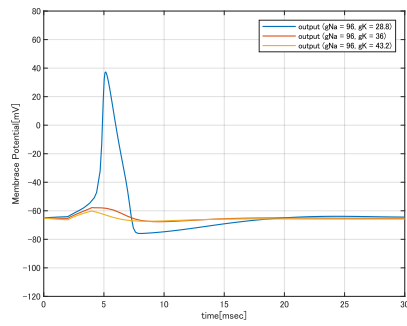


図 12: $g_{Na} = 96$

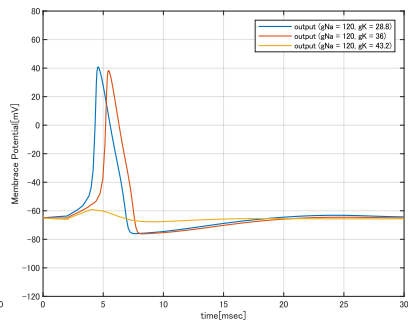


図 13: $g_{Na} = 120$

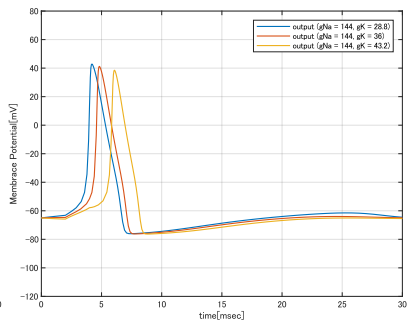


図 14: $g_{Na} = 144$

図 15: g_{Na} に注目した図

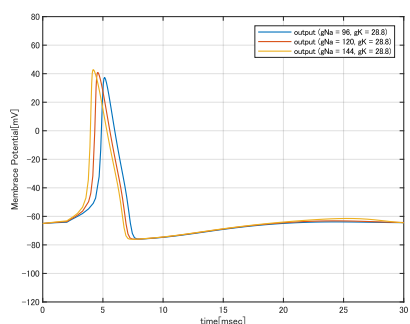


図 16: $g_K = 28.8$

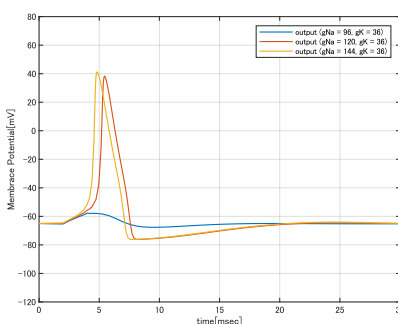


図 17: $g_K = 36$

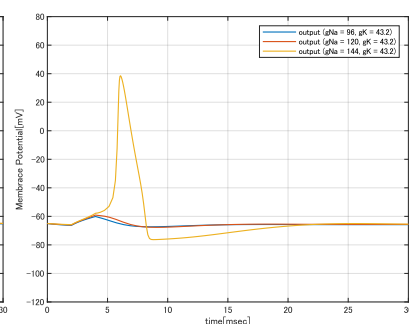


図 18: $g_K = 43.2$

図 19: g_K に注目した図

3.5 考察

$\overline{g_{Na}}$ 、 $\overline{g_K}$ は立ち上がりと電位の頂点 V の高さに関連するパラメータであるようだ。図 15 と図 19 から、 $\overline{g_{Na}}$ が高いほど V の立ち上がりが急勾配であり最高の V の値が高く、立ち下がりには特に変化している様子はないが、結果的にオーバーシュートのタイミングが早くなる。一方、 $\overline{g_K}$ が高いほど、立ち下がり以外の先の文の全てが逆となるからである。余談だが、図 12、図 13、図 14 を比較すると、ある $\overline{g_{Na}}$ に対してオーバーシュート可能な $\overline{g_K}$ の閾値が存在すると考えられる。色々確かめたが、 $\overline{g_K} = 36$ に対して、オーバーシュート ($V \geq 0$) となるような $\overline{g_{Na}}$ の値の閾値は約 $101.0890 \leq \overline{g_{Na}} \leq 101.0897$ のどこかの値であった。その検証の際に、その範囲で電位が段々と変化するような様子はなかったため、詳細に求められなかった閾値の前後で、電位の形状が大きく変化していると考えられる。

3.6 感想

dat ファイルの名前付けを間違えて、再度データ作りを実行したが、その際に入力していた値を間違っていたことに気付いたため、結果的に良かった。

4 実験項目 4

次の条件で入力刺激電流 I_{ext} を加えた際のスパイク応答を観測、記録せよ。実験結果から、スパイクの

表 1

シミュレーション終了時刻	500[ms]
刺激開始時刻	50[ms]
刺激終了時刻	450[ms]
刺激形状	ステップ
刺激強度	0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 6.1, 6.2, 6.25, 6.3, 6.4, 6.5, 8.0, 9.0, 10.0[$\mu A/cm^2$] (その他、実験者が必要と判断した任意の刺激強度)

発火頻度を求めよ、発火頻度は 1 秒間あたりのスパイク数であり、単位は [Hz] である。横軸を刺激強度、縦軸を発火頻度としたグラフを作成せよ。

4.1 概要

刺激強度とスパイクの発火頻度の関係性について、データを収集し、それらを纏めた dat ファイル、pdf 資料を作成した。その pdf と dat のデータに基づいて考察することができた。

4.2 目的

刺激強度に対するスパイクの発火頻度の関係性を検証する。

4.3 方法

図 3 の 2 つの step の最終値を変更しシミュレーションを行う。sum プラス側のステップ値に 50 を、マイナス側のそれには 450 を入れる。この際に、V の線を分岐し、simout でデータを出力し、matlab プログラムのコード 3 を用いて重ね描き plot する。

source 3: dat から折れ線グラフを生成するコード

```
filename = 'kkadai14';
SimDataA = load([filename '.dat'], '-ascii');

figure
plot(SimDataA(:,1), SimDataA(:,2), '-bo', 'Linewidth', 1.0)
xlabel('Stimulus intensity [\mu A/cm^2]')
ylim([0 60])
ylabel('firing rate [Hz]')
legend('reference', ...
       'Location', 'northwest', ...
       'Orientation', 'horizontal', 'NumColumns', 1)
set(findobj('type','axes'),'fontsize',8)
grid on
print('-dpdf', [filename '.pdf'])
```

4.4 結果

表 2 を見れば解かるが、手作業で dat ファイルを作成し、コード 3 を用いて、図 20 を作成した。

表 2: 刺激強度と発火頻度を纏めた表

刺激強度 [$\mu A/cm^2$]	発火頻度 [Hz]
0.000000e+00	0.000000e+00
1.000000e+00	0.000000e+00
2.000000e+00	0.000000e+00
2.100000e+00	0.000000e+00
2.250000e+00	2.000000e+00
2.500000e+00	2.000000e+00
3.000000e+00	2.000000e+00
4.000000e+00	2.000000e+00
5.000000e+00	2.000000e+00
5.500000e+00	2.000000e+00
5.750000e+00	2.000000e+00
5.875000e+00	2.000000e+00
6.000000e+00	4.000000e+00
6.100000e+00	4.000000e+00
6.200000e+00	6.000000e+00
6.225000e+00	8.000000e+00
6.250000e+00	1.000000e+01
6.260000e+00	1.400000e+01
6.262000e+00	1.600000e+01
6.264000e+00	1.800000e+01
6.268000e+00	2.800000e+01
6.270000e+00	4.200000e+01
6.300000e+00	4.200000e+01
6.400000e+00	4.400000e+01
6.500000e+00	4.400000e+01
6.750000e+00	4.600000e+01
7.000000e+00	4.800000e+01
7.500000e+00	5.000000e+01
8.000000e+00	5.000000e+01
8.500000e+00	5.200000e+01
9.000000e+00	5.400000e+01
9.500000e+00	5.400000e+01
1.000000e+01	5.600000e+01
1.050000e+01	5.600000e+01
1.100000e+01	5.800000e+01
1.150000e+01	5.800000e+01
1.200000e+01	6.000000e+01

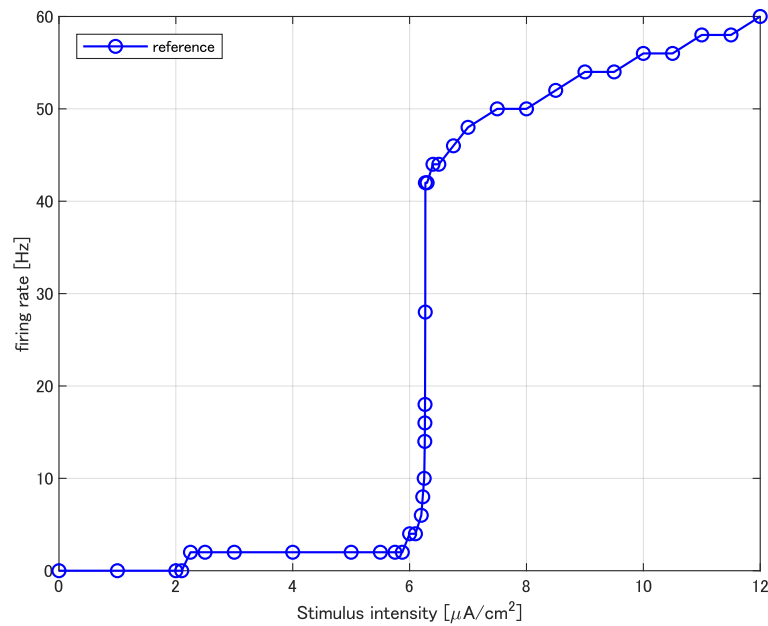


図 20: 刺激強度と発火頻度のグラフ

4.5 考察

図 20 と表 2 から、刺激強度が 6.26 近辺で発火頻度が急激に増加していることが判る。刺激強度 ($0.0 \leq$ 刺激強度 ≤ 2.1) の区間では、発火頻度が 0 である。これは、弱い刺激では神経が電位による情報伝達を行わないことで、例として、人間の皮膚に羽が触れた程度の刺激を伝達し、脳が過大に評価しない為など、微小な電位変化による誤動作を抑制する仕組みなのではないか。加えて、刺激強度 ($2.0 \leq$ 刺激強度 ≤ 5.875) の区間で、発火頻度が 2.0 であるが、シミュレーションの終了時刻が 500[ms] であるため実際の立ち上がり回数 1 回に対する発火は 1 回となり、ある特定の神経細胞が他の無数の神経細胞に対し信号を伝達する際に、1 対 1 で信号の input/output が行われ、脳に届く信号回数が指数関数的 (介した神経細胞数乗) にならない区間であると考えられるのではないか。さらに、刺激強度が強いほど、発火する回数も多く、上記の指数関数的に増大する発火 (信号) 回数により、ある程度長い時間、脳に伝達される仕組みになっていると考えられる。また、認知神経科学研究者の 北城 圭一 氏がウェブサイト Wikipedia に記載されている内容によると、神経筋が複雑な情報を伝達するためには複数の神経筋やそれらの発火頻度による確率分布を用いるという [1]。そのため、神経自体が一本の導線という訳ではなく、複数の筋の纏まりと考えられるのではないか。

- ニューロンモデルの情報の符号化について 刺激の信号時間や、神経を構成する神経細胞の筋の内どれだけの数が発火したか、といった情報を脳が処理しているのではないか。
- 数値積分法や積分の刻み幅の影響について 今回使用した積分法が ode45 であり、Matlab によると、この積分法はノンステップ向けであることが判る [2]。Wikipedia によると固い (ステップ) 常微分方程式の数値積分法は、積分の刻み幅を可能な限り小さくしないと数値的不安定になるという [3]。

実際に、Simulink で最大ステップサイズ (刻み幅) を 0.001 ~ 5.0 で変動させたり、ode45 以外のノンステップな積分法 ode23、ode113 や、ステップな積分法 ode23s、ode15s、ode23t、ode23tb を試したところ、電位のグラフの形状に目だった変化は見られなかったため、この実験の常微分方程式はステップであると考えられるのではないかと。ここで、ode $\bullet$ $\square$ の $\bullet$ や $\square$ には用いるルンゲクッタ法の次数が入る。さくらのレンタルサーバというウェブサイトの SIKINO 氏によると、ルンゲクッタ法は次数が高いほど精度が高まり、計算コストもそれと同様の次数オーダー O かそれ以上 (次数 5 以降) に増大するという [5]。再度 Matlab から、ode113 は 1 次から 13 次項までを用いるといい、許容誤差が厳しい場合に ode45 よりも好ましい場合があるという [4]。先ほどの引用はこの理由付けであり、加えて Matlab から、ode23 は計算コストは ode45 と比較して計算コストも精度も低いことの理由でもある [2]。また、ode $\bullet$ $\square$ の仕組みも考慮する。Qiita で taka_horibe 氏が解説されている内容によると、ode45 は陽的 4 次ルンゲクッタ法と陽的 5 次ルンゲクッタ法を利用して、より具体的には、前者と実際の積分値との誤差が最小となるように、後者の計算過程で無視される (後者 + 1) 次の項を最小にする様にステップ幅を定めているという [6]。これまでの内容から、今回の実験では行わなかったが、最大ステップサイズではなく最小ステップサイズを auto ではないなんらかの値に調整すれば、ソルバーの結果に差が得られた可能性も否定できないのではないかと。また、終了時間が 500 であったことも結果の差が見られなかった原因の可能性も十分考えられる。

4.6 感想

刺激強度が高まるにつれて、オーバーシュートした電位の山を数えるのが大変になった。

参考文献

- [1] 北城 圭一: "神経符号化", Wikipedia. 2021, <https://bsd.neuroinf.jp/wiki/%E7%A5%9E%E7%B5%8C%E7%AC%A6%E5%8F%B7%E5%8C%96%E7%AC%A6%E5%8F%B7%E5%8C%96>, (参照 2024-5-16).

- [2] Matlab: "ODE ソルバーの選択 - MATLAB & Simulink", Matlab. 2024,
<https://jp.mathworks.com/help/matlab/math/choose-an-ode-solver.html>, (参照 2024-5-21).
- [3] Wikipedia: "硬い方程式", Wikipedia. 2023,
<https://ja.wikipedia.org/wiki/%E7%A1%AC%E3%81%84%E6%96%B9%E7%A8%8B%E5%BC%8F>, (参照 2024-5-21).
- [4] Matlab: "MATLAB ode113 - ノンスティッフ微分方程式の求解", Matlab. 2024,
<https://jp.mathworks.com/help/matlab/ref/ode113.html>, (参照 2024-5-21).
- [5] SIKINO: "ルンゲ=クッタ法の説明と刻み幅制御 — (更新停止)", さくらのレンタルサーバ. 2015,
<https://slpr.sakura.ne.jp/qp/runge-kutta-ex/#rk4reason>, (参照 2024-5-21).
- [6] taka_horibe: "微分方程式を数値計算するときの話 (ode45 の中身) MATLAB", Qiita. 2022,
https://qiita.com/taka_horibe/items/8838b08f88f1a55fb9a7, (参照 2024-5-21).